

🌟 CSS Image Shapes

Create complex shapes and wrap text around images with CSS

What are CSS Image Shapes?

CSS Image Shapes allow you to create non-rectangular layouts and wrap text around complex shapes. This is achieved using two main techniques: **clip-path** for clipping images into shapes, and **shape-outside** for wrapping text around shapes. Together, they enable creative layouts that were previously only possible with images or JavaScript.

Common uses include circular profiles, creative hero sections, magazine-style layouts, and artistic web designs.

Browser Support: **clip-path** is widely supported in all modern browsers. **shape-outside** is supported in Chrome, Firefox, and Safari, but IE/Edge (legacy) have limited support.

Core Concepts

Two Main Techniques

CSS provides two primary methods for working with image shapes:

- clip-path** - Clips an element into a specific shape, hiding content outside the path
- shape-outside** - Defines how text wraps around a shaped element

Understanding clip-path

clip-path visually masks an element to a specific shape. The content outside the clipped area is hidden but still takes up space in the DOM.

Syntax: clip-path: <clip-path-value>;

Values:

- polygon(x1 y1, x2 y2, ...)
- circle(radius)
- ellipse(rx ry)
- inset(top right bottom left)
- path(svg path)
- url(#clip-path-id)

Understanding shape-outside

shape-outside defines how inline content (text, images) flows around a floated element. Only works on floated elements.

Syntax: shape-outside: <shape-value>;

Common Values:

- circle(radius at x y)
- ellipse(rx ry at x y)
- polygon(x1 y1, x2 y2, ...)
- url(image-url)

Key Difference

clip-path is for the shape itself, while **shape-outside** is for how content flows around it.

clip-path - Creating Shapes

1. polygon()

Defines a shape using coordinate points. Most flexible for custom shapes.

Syntax: clip-path: polygon(x1 y1, x2 y2, x3 y3, ...);

Coordinates: Percentages (0% to 100%) or pixels

Examples:

```
/* Triangle */
clip-path: polygon(50% 0%, 100% 100%, 0% 100%);

/* Pentagon */
clip-path: polygon(50% 0%, 100% 38%, 82% 100%, 18% 100%, 0% 38%);

/* Custom shape */
clip-path: polygon(20% 0%, 100% 0%, 90% 100%, 10% 100%);
```

2. circle()

Creates a circular shape. Perfect for profile images and circular layouts.

Syntax: clip-path: circle(radius [at position]);

Parameters:

- radius: Size of circle (px, %, or closest-side, farthest-side)
- position: Center point (optional, default 50% 50%)

Examples:

```
clip-path: circle(50%);
clip-path: circle(100px);
clip-path: circle(50% at 25% 75%);
```

3. ellipse()

Creates an oval/elliptical shape with different radii for x and y axes.

Syntax: clip-path: ellipse(rx ry [at position]);

Parameters:

- rx, ry: Radii on x and y axes
- position: Center point (optional)

Examples:

```
clip-path: ellipse(75% 50%);
clip-path: ellipse(100px 50px);
clip-path: ellipse(50% 75% at 25% 50%);
```

4. inset()

Creates a rectangular shape with optional rounded corners by cropping from edges.

Syntax: clip-path: inset(top right bottom left [round radius]);

Parameters:

- top, right, bottom, left: Distance to crop from each edge
- round: Optional border-radius value

Examples:

```
clip-path: inset(10% 5% 10% 5%);
clip-path: inset(20px 10px 20px 10px round 15px);
clip-path: inset(0% 0% 0% 0% round 50%);
```

5. path()

Uses SVG path syntax for precise shape control.

Syntax: clip-path: path('SVG path data');

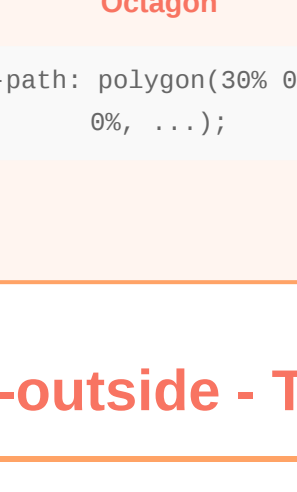
Example:

```
clip-path: path('M10 10 h 100 v 100 h -100 z');
```



Interactive Shape Demos

Explore different clip-path shapes and see how they transform elements.



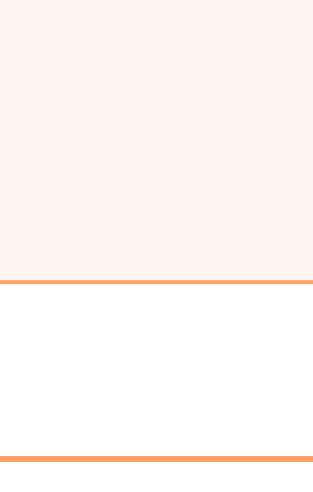
Circle

```
clip-path: circle(50%);
```



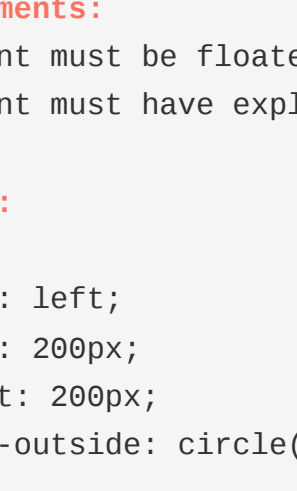
Ellipse

```
clip-path: ellipse(75% 50%);
```



Triangle

```
clip-path: polygon(50% 0%, 100% 100%, 0% 100%);
```



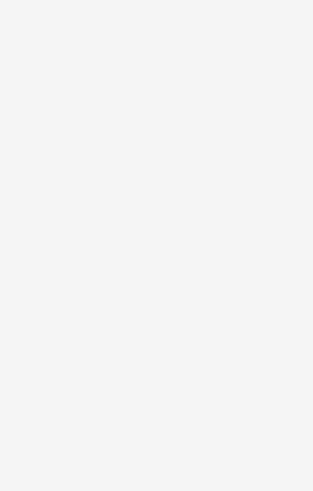
Pentagon

```
clip-path: polygon(50% 0%, 100% 38%, ...);
```



Hexagon

```
clip-path: polygon(50% 0%, 100% 25%, ...);
```



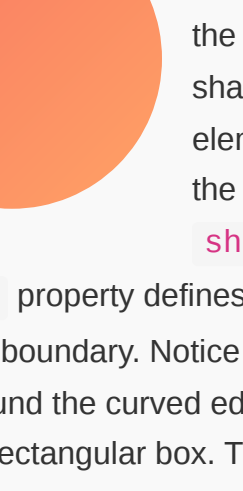
Star

```
clip-path: polygon(50% 0%, 61% 35%, ...);
```



Octagon

```
clip-path: polygon(30% 0%, 70% 0%, ...);
```



Inset Rounded

```
clip-path: inset(20% 10% ... round 15px);
```

shape-outside - Text Wrapping

How shape-outside Works

The **shape-outside** property only works on floated elements. It defines how content flows around the shape.

Syntax: shape-outside: <shape>;

Requirements:

- Element must be floated (float: left or float: right)
- Element must have explicit width and height

Example:

```
img {
  float: left;
  width: 200px;
  height: 200px;
  shape-outside: circle(50%);
}
```

Text Wrapping Example

This text wraps around the circular shaped element on the left. The **shape-outside** property defines the wrapping boundary. Notice how text flows around the curved edges rather than the rectangular box. This creates more natural, magazine-style layouts.

shape-outside with Different Shapes

```
/* Circle - Text wraps around circle */
shape-outside: circle(50%);

/* Ellipse - Text wraps around oval */
shape-outside: ellipse(75% 50%);

/* Polygon - Text wraps around custom shape */
shape-outside: polygon(0% 0%, 100% 0%, 75% 100%, 0% 100%);

/* Image - Text wraps around image alpha channel */
shape-outside: url('image.png');
```

Combining clip-path and shape-outside

For best results, use both properties together: **clip-path** creates the visual shape, while **shape-outside** controls text wrapping.

```
img {
  float: left;
  width: 200px;
  height: 200px;
  clip-path: circle(50%); /* Visual shape */
  shape-outside: circle(50%); /* Text wrapping */
  shape-margin: 10px; /* Padding around shape */
}
```

Common Use Cases & Patterns

Circular Profile Images

```
.profile-image {
  width: 150px;
  height: 150px;
  clip-path: circle(50%);
  box-shadow: 0 5px 15px rgba(0,0,0,0.2);
}
```

Magazine-Style Text Wrap with Circle

```
.article-image {
  float: left;
  width: 250px;
  height: 250px;
  margin-right: 30px;
  clip-path: circle(50%);
  shape-outside: circle(50%);
  shape-margin: 15px;
}
```

Diagonal Image with Polygon

```
.diagonal-image {
  clip-path: polygon(0% 0%, 100% 0%, 100% 85%, 0% 100%);
  width: 100%;
  height: auto;
}
```

Irregular Shape with Polygon and Text Wrap

```
.irregular-image {
  float: left;
  width: 200px;
  height: 200px;
  clip-path: polygon(25% 0%, 100% 0%, 75% 100%, 0% 100%);
  shape-outside: polygon(25% 0%, 100% 0%, 75% 100%, 0% 100%);
}
```

Image with Rounded Corners

```
.rounded-image {
  clip-path: inset(0% round 20px);
  /* Or use modern approach: */
  border-radius: 20px;
}
```

Hero Section with Curved Edge

```
.hero-image {
  clip-path: polygon(0% 0%, 100% 0%, 100% 90%, 0% 100%);
  width: 100%;
  height: 500px;
  object-fit: cover;
}
```

Artistic Gallery Image

```
.gallery-image {
  clip-path: polygon(10% 0%, 90% 0%, 100% 10%, 100% 90%, 90% 100%, 10% 100%, 0% 90%, 0% 10%);
  transition: clip-path 0.3s ease;
}

.gallery-image:hover {
  clip-path: polygon(0% 0%, 100% 0%, 100% 100%, 0% 100%);
}
```

Advanced Techniques

Animated clip-path Transitions

```
.image {
  clip-path: circle(0%);
  transition: clip-path 0.5s ease;
}

.image:hover {
  clip-path: circle(50%);
}
```

Using CSS Variables with clip-path

```
:root {
  --clip-amount: 50%;
}

img {
  clip-path: circle(var(--clip-amount));
}

img:hover {
  --clip-amount: 30%;
}
```

shape-margin for Spacing

The **shape-margin** property adds space around the shape boundary for text wrapping.

```
.float-image {
  float: left;
  shape-outside: circle(50%);
  shape-margin: 20px; /* Adds 20px padding around shape */
}
```

Multiple Shapes with SVG path()

```
.custom-shape {
  clip-path: path('M 10 10 L 100 10 L 100 100 L 10 100 Z');
}
```

Browser Compatibility

clip-path: Supported in Chrome, Firefox, Safari, Edge (95+). Older Edge and IE do not support it.

shape-outside: Supported in Chrome, Firefox, Safari. Limited support in Edge.

Vendor Prefixes

```
/* Safari may need -webkit prefix in older versions */
-webkit-clip-path: circle(50%);
clip-path: circle(50%);
```

Best Practices

- **Performance:** Avoid complex clip-paths on many elements. Rendering performance depends on complexity.
- **Accessibility:** Clipped content is still in the DOM and accessible to screen readers. No content is lost.
- **Combine Techniques:** Use both **clip-path** (visual) and **shape-outside** (text wrap) together for best results.
- **Test in Browsers:** Always test in target browsers. Feature support varies.
- **Use clip-path for Hover:** Smooth transitions work well with clip-path for interactive effects.
- **shape-margin:** Always add margin around shapes for better text readability.
- **Fallbacks:** For unsupported browsers, images will display as rectangles. Plan accordingly.
- **SVG Path:** Use SVG path() for complex custom shapes. Keep paths simple for performance.

Property/Function	Purpose	Example
clip-path: circle()	Creates circular shape	circle(50%)
clip-path: ellipse()	Creates oval shape	ellipse(75% 50%)
clip-path: polygon()	Creates custom polygon	polygon(50% 0%, 100% 100%, 0% 100%)
clip-path: inset()	Rectangular crop with radius	inset(20% round 15px)
clip-path: path()	SVG path-based shape	path("M10 10 L100 10 L100 100 Z")
shape-outside	Controls text wrap around floated element	circle(50%)
shape-margin	Adds space around shape for text	20px

Common Mistakes to Avoid

- **Forgetting to specify dimensions:** For **shape-outside** to work, element must have explicit width/height.
- **Using shape-outside without float:** **shape-outside** only works on floated elements.
- **Not combining clip-path and shape-outside:** Use both together for consistent visual/text behavior.
- **Complex polygons causing performance issues:** Keep polygon points reasonable. Too many points can cause lag.
- **Using coordinates outside 0-100%:** Coordinates should stay within element bounds (0-100% or 0-width).
- **Forgetting shape-margin:** Without margin, text wraps too close to the shape edge.
- **Not testing fallbacks:** Older browsers ignore clip-path. Test how layout looks in rectangular fallback.
- **Incorrect polygon winding order:** Points should be in order (clockwise or counter-clockwise). Reversed order can produce unexpected results.

Polygon Coordinates Explained

Understanding coordinates is key to creating custom polygons. The element is treated as a 100% × 100% box, with (0% 0%) at the top-left corner.

Coordinate System:

```
(0%, 0%) ————— (100%, 0%)
|
|
|
(0%, 100%) ————— (100%, 100%)
```

Example - Triangle points to three corners:

```
polygon(50% 0%, 100% 100%, 0% 100%)
• Point 1: Top center (50% 0%)
• Point 2: Bottom right (100% 100%)
• Point 3: Bottom left (0% 100%)
```